

# Лабораторная работа № 7

## Соединение и тестирование клиентской и серверной частей веб-приложения

### 1. Место работы в курсе

К началу седьмой работы StudySpace содержит адаптивный интерфейс из ЛР1–ЛР4, локальный PostgreSQL и Prisma ORM из ЛР5, а также физические таблицы, seed и серверную авторизацию из ЛР6. Однако клиентские компоненты всё ещё могут читать статический массив, хранить избранное в `useState` и использовать `DemoAuthProvider`.

В заключительной работе клиент переводится на реальные Route Handlers. Галерея получает StudyPlace из PostgreSQL, AuthProvider восстанавливает пользователя через `/api/auth/me`, формы регистрации и входа отправляют запросы серверу, а избранное сохраняется в FavoritePlace. После соединения выполняются проверки на трёх уровнях: ручные HTTP-сценарии, модульные тесты чистой логики с Vitest и сквозной пользовательский сценарий с Playwright.

В TheraCity клиент и сервер соединены большим числом маршрутов и сущностей. StudySpace воспроизводит архитектурный принцип и методику тестирования на собственной предметной области, не копируя интерфейс, данные и бизнес-правила TheraCity.

### 2. Цель работы

Соединить клиентские компоненты StudySpace с серверными Route Handlers и PostgreSQL, заменить демонстрационное состояние пользователя и избранного реальными запросами, реализовать состояния загрузки и ошибок, проверить HTTP-контракты и авторизацию, добавить модульные и end-to-end тесты и выполнить итоговую проверку всего приложения.

### 3. Задачи работы

В ходе выполнения работы студент должен:

- определить HTTP-контракт между клиентом и сервером;
- реализовать GET `/api/places`;
- преобразовать Decimal-координаты в JSON-совместимые значения;
- добавить isFavorite относительно текущего пользователя;
- заменить DemoAuthProvider на AuthProvider;
- восстанавливать сессию через GET `/api/auth/me`;
- подключить формы регистрации, входа и выхода к API;
- получать галерею из PostgreSQL, а не из статического массива;
- реализовать loading, error и empty состояния;
- сохранять избранное через POST и DELETE;

- обрабатывать 400, 401, 404, 409 и 500 на клиенте;
- не доверять userId и isFavorite, присланным клиентом;
- сохранить модальное окно, карту, меню и адаптивность предыдущих работ;
- проверить запросы во вкладке Network;
- проверить устойчивость состояния после перезагрузки страницы;
- установить и настроить Vitest;
- написать модульные тесты нормализации и валидации;
- установить и настроить Playwright;
- написать сквозной сценарий регистрации и избранного;
- обеспечить очистку тестового пользователя;
- выполнить prisma migrate status, lint, unit tests, build и e2e tests;
- подготовить итоговую матрицу проверок курса.

## 4. Теоретическая часть

### 4.1 Клиент-серверный поток данных

В полном веб-приложении клиентский компонент не обращается напрямую к файлам PostgreSQL. Браузер отправляет HTTP-запрос Route Handler, сервер проверяет входные данные и пользователя, Prisma Client выполняет запрос к PostgreSQL, а сервер возвращает JSON-ответ. Клиент преобразует ответ в состояние React и обновляет интерфейс.

Для StudySpace поток чтения выглядит так:

```
PlacesGallery → GET /api/places → Prisma Client → PostgreSQL
                ← JSON places + isFavorite ←
```

Поток изменения избранного:

```
кнопка → POST/DELETE /api/favorites/:placeId → проверка cookie и User
        → upsert/deleteMany FavoritePlace → JSON → обновление карточки
```

Названия маршрутов и форма DTO являются контрактом курса, а правила Request, Response и Route Handlers основаны на официальной документации.

### 4.2 Route Handlers в App Router

Route Handlers определяются файлом route.ts внутри app и используют Web Request и Response API. Next.js поддерживает GET, POST, PUT, PATCH, DELETE, HEAD и OPTIONS. Неподдерживаемый метод получает 405. Route Handlers доступны только в app и не должны находиться на одном сегменте с page.tsx. ([Next.js: Route Handlers](#))

```
// Возвращаем JSON из GET-обработчика.
export async function GET() {
  return Response.json({ ok: true });
}
```

GET Route Handler по умолчанию не кэшируется в актуальном App Router, но при чтении cookie маршрут в любом случае зависит от конкретного запроса. Для учебной ясности `/api/places` объявляется `dynamic = "force-dynamic"`.

## 4.3 HTTP-контракт и методы

HTTP-метод выражает намерение запроса. В StudySpace используются:

- GET — чтение мест и текущего пользователя;
- POST — регистрация, вход, выход и добавление избранного;
- DELETE — удаление избранного.

Ответ включает status и JSON-тело. Успешный контракт курса:

- 200 OK — чтение, вход, выход или удаление;
- 201 Created — регистрация или создание связи избранного.

Ошибки:

- 400 Bad Request — неверная форма входных данных;
- 401 Unauthorized — отсутствует подтверждённая сессия;
- 404 Not Found — место не существует;
- 409 Conflict — нарушена уникальность email;
- 500 Internal Server Error — непредвиденная серверная ошибка.

Статусы не заменяют понятное безопасное сообщение, а сообщение не заменяет корректный статус.

## 4.4 DTO и сериализация

Data Transfer Object — объект, специально сформированный для передачи через границу сервера. Он не обязан повторять всю строку базы. Руководство Next.js по авторизации рекомендует возвращать только необходимые данные, чтобы чувствительные поля не попали клиенту. ([Next.js: Authentication](#))

UserDTO содержит id, name и email, но не passwordHash. StudyPlaceDTO содержит поля карточки, координаты number и вычисленный isFavorite. createdAt не отправляется, если интерфейс его не использует.

Prisma Decimal преобразуется явно:

```
// Преобразуем координаты в обычные JSON-числа для карты.  
latitude: Number(place.latitude),  
longitude: Number(place.longitude),
```

Сервер вычисляет isFavorite на основе проверенного user.id. Клиент не может назначить чужой userId и не является источником истины для авторизации.

## 4.5 Fetch API, JSON и cookie

Fetch API возвращает Promise объекта Response. Ошибочный HTTP-статус сам по себе не отклоняет Promise, поэтому клиент обязан проверить `response.ok` и прочитать тело. ([MDN: Fetch API](#))

```
// Проверяем статус до использования JSON как успешного результата.  
const response = await fetch("/api/places", { cache: "no-store" });  
const payload = await response.json();  
if (!response.ok) throw new Error("Не удалось загрузить места");
```

Для `same-origin` запроса cookie отправляются по умолчанию, но в учебном коде `credentials: "same-origin"` указывается явно. `httpOnly` cookie нельзя прочитать через `document.cookie`; браузер прикладывает её к запросу, а сервер читает через `cookies()`.

## 4.6 Состояния асинхронного интерфейса

Сетевой запрос не завершается мгновенно и может завершиться ошибкой или пустым результатом. Поэтому интерфейс должен различать:

- `loading` — запрос выполняется;
- `success` — данные получены;
- `empty` — успешный ответ не содержит мест;
- `error` — запрос или ответ завершился неуспешно.

Показывать пустую сетку во всех четырёх случаях нельзя: пользователь не понимает, загружается ли страница, действительно ли данных нет или сервер недоступен.

В `useEffect` создаётся `AbortController`. `Cleanup` отменяет запрос при размонтировании компонента и предотвращает обновление уже неактуального состояния.

## 4.7 Синхронизация AuthProvider

React Context работает только в Client Components. Руководство Next.js отмечает, что контекст не доступен Server Components; он подходит для клиентского состояния интерфейса, но серверные проверки всё равно должны выполняться рядом с источником данных. ([Next.js: Authentication](#))

`AuthProvider` хранит три состояния:

- `loading` — GET `/api/auth/me` ещё выполняется;
- `authenticated` — сервер вернул пользователя;
- `guest` — сервер вернул `user: null`.

Методы `login` и `register` сначала получают подтверждённый `UserDTO` от сервера, затем обновляют контекст. Метод `logout` удаляет cookie через `Route Handler` и очищает `user`. После перезагрузки `refresh` снова получает состояние с сервера.

## 4.8 Пессимистичное и оптимистичное обновление

При пессимистичном обновлении клиент ждёт успешный ответ сервера и только затем меняет UI. При оптимистичном обновлении UI меняется сразу, а при ошибке выполняется откат.

В лабораторной работе избранное обновляется пессимистично. Этот вариант проще проверять: кнопка блокируется, запрос завершается, сервер подтверждает изменение, после чего `isFavorite` меняется. Для улучшения ощущения скорости можно перейти на оптимистичную модель, но тогда обязательны откат, защита от повторных кликов и обработка конфликтов.

## 4.9 Авторизация остаётся серверной

Клиент может скрыть кнопку для гостя, но защищённый Route Handler всё равно вызывает `getCurrentUser`. Официальное руководство Next.js рекомендует проверять авторизацию перед изменением данных и располагать безопасные проверки близко к источнику данных. ([Next.js: Authentication](#))

Сервер использует `user.id` из проверенной cookie-сессии. Поле `userId` не принимается из JSON или URL. Это предотвращает изменение `FavoritePlace` другого пользователя через подмену клиентского запроса.

## 4.10 Виды тестирования

Официальное руководство Next.js различает:

- unit testing — отдельная функция или небольшой блок в изоляции;
- component testing — поведение компонента и его props;
- integration testing — совместная работа нескольких частей;
- end-to-end testing — пользовательский сценарий в браузере, близком к реальному;
- snapshot testing — сравнение вывода со снимком. ([Next.js: Testing](#))

В работе чистые функции `normalizeEmail` и `validateRegistration` тестируются модульно. Полный путь «страница → Route Handler → Prisma → PostgreSQL → cookie → страница» проверяется Playwright. Next.js рекомендует E2E-тестирование вместо unit-тестирования `async Server Components`, поддержка которых тестовыми инструментами может быть ограничена.

## 4.11 Vitest

Vitest — тестовый фреймворк, который поддерживает TypeScript и ищет файлы `.test.` или `.spec.` По официальной документации `test` задаёт проверку, а `expect` формулирует ожидаемый результат. Команда `vitest run` выполняет тесты один раз, что удобно для итоговой проверки. ([Vitest: Getting Started](#); [Writing Tests](#))

Модульные тесты должны быть детерминированными: они не зависят от PostgreSQL, времени, сети и порядка запуска. Поэтому в них проверяется чистая валидация, а не реальная регистрация.

## 4.12 Playwright

Playwright запускает браузер и автоматизирует пользовательские действия. Конфигурация `webServer` может запустить локальный сервер перед тестами, а `baseUrl` позволяет использовать относительные адреса. `reuseExistingServer` обычно разрешают локально и отключают в CI. ([Playwright: Web server](#))

Для более близкой к production проверки сначала выполняется `npm run build`, а Playwright запускает `npm run start`. Тест использует доступные селекторы `getByRole` и `getByLabel` вместо нестабильных CSS-классов.

## 4.13 Тестовые данные и изоляция

Тест не должен зависеть от заранее свободного email. Он создаёт адрес с `Date.now()`, а `afterAll` удаляет тестового пользователя. Внешние ключи `FavoritePlace` с `onDelete: Cascade` удаляют зависимые связи автоматически.

Для командного или production-проекта следует использовать отдельную test-базу и воспроизводимую миграцию. В учебной локальной работе допускается development-база при обязательной очистке. Нельзя запускать тест, который удаляет все строки или сбрасывает базу без точной проверки окружения.

## 4.14 Итоговая стратегия проверки

Один вид теста не заменяет остальные:

- `prisma validate` и `migrate status` проверяют конфигурацию и историю схемы;
- `lint` проверяет правила статического анализа;
- `unit tests` быстро проверяют чистую логику;
- `build` проверяет типы и production-сборку Next.js;
- `e2e` проверяет реальный пользовательский поток;
- ручная проверка дополняет автоматизацию картой, модальным окном, адаптивностью и доступностью.

Проходящий E2E-сценарий не доказывает отсутствие всех ошибок безопасности. Тест подтверждает только сформулированный сценарий и ожидаемые проверки.

# 5. Практическое задание

## Этап 1. Подготовка исходной точки

Проверьте ЛР6:

- `prisma migrate status` сообщает об актуальной схеме;
- seed-данные присутствуют;
- регистрация, вход, те и выход работают;
- защищённый маршрут избранного возвращает 401 гостю;
- `npm run lint` и `npm run build` проходят.

Создайте резервную ветку. Не удаляйте статический `app/data/places.ts` до успешного подключения API: он понадобится для сравнения полей.

## Этап 2. Фиксация HTTP-контракта

До изменения компонентов запишите форму ответов:

```
GET /api/places          → { places: StudyPlaceDTO[] }
GET /api/auth/me         → { user: UserDTO | null }
POST /api/auth/login     → { user: UserDTO }
POST /api/auth/register → { user: UserDTO }
POST /api/auth/logout    → { ok: true }
POST /api/favorites/:id → { favorite: ... }
DELETE /api/favorites/:id → { deleted: number }
```

Укажите успешные и ошибочные статусы. Изменение контракта должно одновременно отражаться на сервере, клиенте и в тестах.

## Этап 3. Маршрут получения учебных мест

Создайте `GET /api/places` по приложению. Маршрут должен:

- получить `current user` или `null`;
- прочитать `StudyPlace` из `Prisma`;
- прочитать `favorite placeId` только для вошедшего пользователя;
- создать `Set` идентификаторов;
- преобразовать `Decimal`-координаты;
- вернуть `isFavorite`;
- не возвращать лишние поля.

Проверьте JSON в браузере или Network.

## Этап 4. Реальный AuthProvider

Создайте `app/providers/AuthProvider.tsx`. Замените `useDemoAuth` на `useAuth` во всех клиентских компонентах. В `RootLayout` замените `DemoAuthProvider` на `AuthProvider`.

`AuthProvider` должен:

- выполнить `refresh` при первом монтировании;
- хранить `user` и `status`;
- предоставить `register`, `login`, `logout` и `refresh`;
- проверить `response.ok`;
- показывать безопасное сообщение сервера;
- не хранить пароль, JWT или cookie в React state.

## Этап 5. Подключение форм

Обновите страницы `/auth/register` и `/auth/login`. При отправке:

- запретите стандартную перезагрузку формы;
- отключите `submit` на время запроса;
- очистите старую ошибку;
- вызовите метод `AuthProvider`;
- при успехе выполните `router.push("/gallery")`;
- при ошибке покажите сообщение рядом с формой;
- не выводите введенный пароль в `Console`.

Кнопка выхода в меню вызывает `logout`, а затем переводит на `/start` или оставляет пользователя в гостевом состоянии.

## Этап 6. Загрузка галереи из API

Удалите импорт `studyPlaces` из `PlacesGallery`. Опишите `StudyPlaceDTO` и загрузите `/api/places` в `useEffect`.

Реализуйте четыре состояния:

- «Загрузка учебных мест...»;
- сетка карточек;
- «Учебные места пока не добавлены»;
- сообщение ошибки и кнопка «Повторить».

Используйте `AbortController` в `cleanup`. Сохраните `PhotoMap` и `PhotoModal`, передавая им загруженный массив.

## Этап 7. Подключение избранного

Замените локальный `Set favoriteIds` серверным полем `isFavorite`. При клике:

- для гостя покажите ссылку на вход;
- заблокируйте кнопку конкретного места;
- выберите `POST` или `DELETE` по текущему `isFavorite`;
- дождитесь успешного `response`;
- измените только соответствующий объект в `places`;
- при `401` выполните `refresh auth`-состояния;
- при ошибке сохраните прежнее значение.

После добавления перезагрузите страницу. Избранное должно сохраниться, поскольку источник истины — `FavoritePlace` в `PostgreSQL`.

## Этап 8. Согласование модального окна, меню и карты

В PhotoModal замените useDemoAuth на useAuth. Кнопка избранного вызывает тот же server toggle, что и карточка. BurgerMenu показывает имя UserDTO и вызывает logout. PhotoMap получает координаты number из StudyPlaceDTO.

Повторите проверки ЛР3–ЛР4:

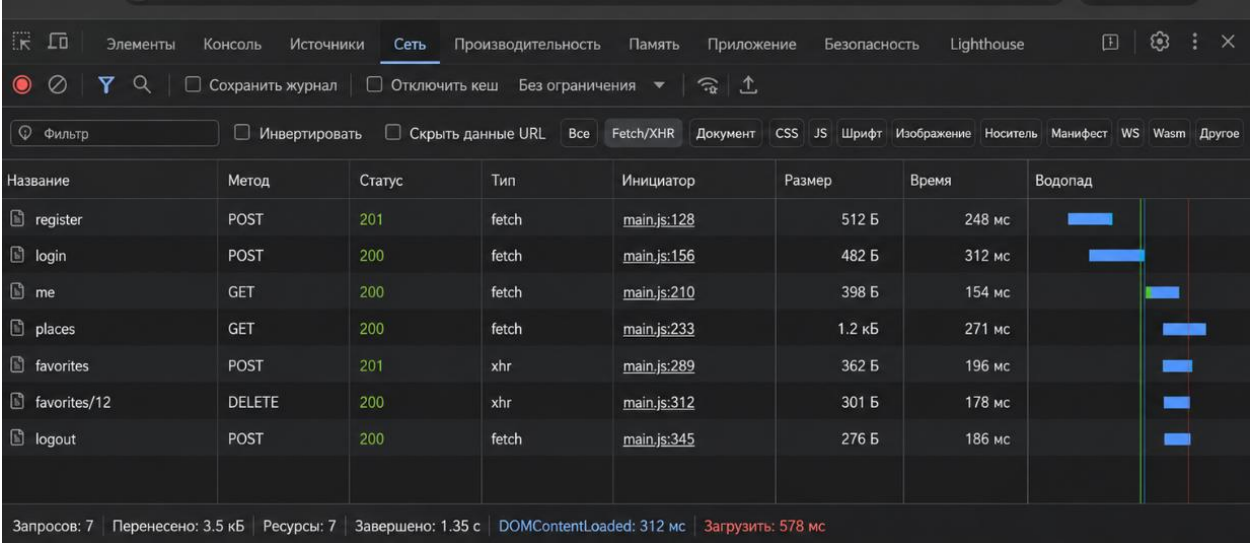
- Escape и цикл Tab в модальном окне;
- возврат фокуса;
- работа меню для гостя и пользователя;
- корректные метки карты;
- отсутствие горизонтальной прокрутки;
- ширины 320, 375, 768, 1024 и 1440 CSS px.

## Этап 9. Проверка Network и PostgreSQL

Во вкладке Network для каждого сценария зафиксируйте method, URL, status, request body без пароля и response body без секретов.

Сопоставьте UI и базу:

```
-- Показываем избранное с понятными пользовательскими данными.
SELECT u.email, p.slug, f."createdAt"
FROM "FavoritePlace" AS f
JOIN "User" AS u ON u.id = f."userId"
JOIN "StudyPlace" AS p ON p.id = f."placeId"
ORDER BY u.email, p.slug;
```



| Название     | Метод  | Статус | Тип   | Инициатор   | Размер | Время  | Водопад |
|--------------|--------|--------|-------|-------------|--------|--------|---------|
| register     | POST   | 201    | fetch | main.js:128 | 512 Б  | 248 мс |         |
| login        | POST   | 200    | fetch | main.js:156 | 482 Б  | 312 мс |         |
| me           | GET    | 200    | fetch | main.js:210 | 398 Б  | 154 мс |         |
| places       | GET    | 200    | fetch | main.js:233 | 1.2 кБ | 271 мс |         |
| favorites    | POST   | 201    | xhr   | main.js:289 | 362 Б  | 196 мс |         |
| favorites/12 | DELETE | 200    | xhr   | main.js:312 | 301 Б  | 178 мс |         |
| logout       | POST   | 200    | fetch | main.js:345 | 276 Б  | 186 мс |         |

Запросов: 7 | Перенесено: 3.5 кБ | Ресурсы: 7 | Завершено: 1.35 с | DOMContentLoaded: 312 мс | Загрузить: 578 мс

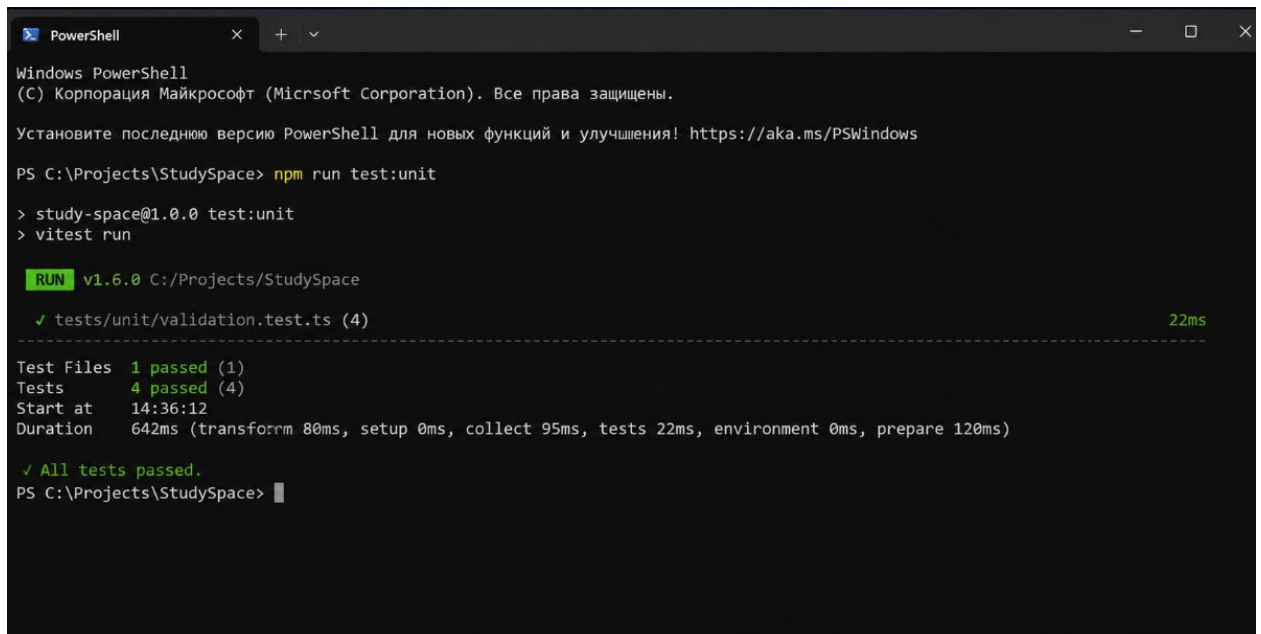
## Этап 10. Установка Vitest

Выполните:

```
# Устанавливаем unit-test runner и поддержку путей из tsconfig.
npm install -D vitest vite-tsconfig-paths
```

Создайте vitest.config.mts и tests/unit/validation.test.ts. Проверьте нормализацию email, корректную регистрацию, короткий пароль и несовпадение паролей.

Запустите npm run test:unit дважды. Результат не должен зависеть от базы.



```
PowerShell
Windows PowerShell
(C) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

Установите последнюю версию PowerShell для новых функций и улучшения! https://aka.ms/PSWindows

PS C:\Projects\StudySpace> npm run test:unit

> study-space@1.0.0 test:unit
> vitest run

 RUN  v1.6.0 C:/Projects/StudySpace

 ✓ tests/unit/validation.test.ts (4) 22ms

Test Files  1 passed (1)
Tests       4 passed (4)
Start at    14:36:12
Duration    642ms (transform 80ms, setup 0ms, collect 95ms, tests 22ms, environment 0ms, prepare 120ms)

✓ All tests passed.
PS C:\Projects\StudySpace>
```

## Этап 11. Установка Playwright

Выполните:

```
# Добавляем Playwright Test и браузер Chromium.
npm install -D @playwright/test
npx playwright install chromium
```

Создайте playwright.config.ts. Конфигурация должна использовать baseURL, один worker для общей локальной базы и webServer с npm run start. Перед E2E обязательно выполните npm run build.

```
Windows PowerShell
PS C:\Projects\StudySpace> npm run test:e2e
> study-space@1.0.0 test:e2e
> playwright test

Running 1 test using 1 worker

[chromium] › tests/e2e/favorites.spec.ts:4:1 › пользователь регистрируется и сохраняет библиотеку в избранное
✓ 1 passed (6.2s)

1 passed
0 failed
0 skipped
Done in 7.10s.

PS C:\Projects\StudySpace> |
```

## Этап 12. Сквозной тест

Создайте tests/e2e/auth-and-favorites.spec.ts. Сценарий должен:

- открыть регистрацию;
- создать уникальный email;
- заполнить доступные поля через label;
- дождаться перехода в галерею;
- добавить библиотеку в избранное;
- перезагрузить страницу;
- подтвердить сохранённое состояние;
- выполнить выход;
- удалить тестового пользователя в afterAll.

Не используйте фиксированную задержку `waitForTimeout`. Playwright автоматически ожидает доступность элементов и поддерживает `web-first assertions`.

## Этап 13. Негативные сценарии

Ручными запросами или отдельными тестами проверьте:

- неверный JSON или пустые поля → 400;
- повторный email → 409;
- неверный пароль → 401;
- изменение избранного гостем → 401;
- неизвестный placeId → 404;
- остановленный PostgreSQL → безопасная 500-ошибка без секрета;
- удалённый User при существующей cookie → `user: null`.

После теста снова запустите PostgreSQL и восстановите рабочее состояние.

## Этап 14. Итоговая последовательность

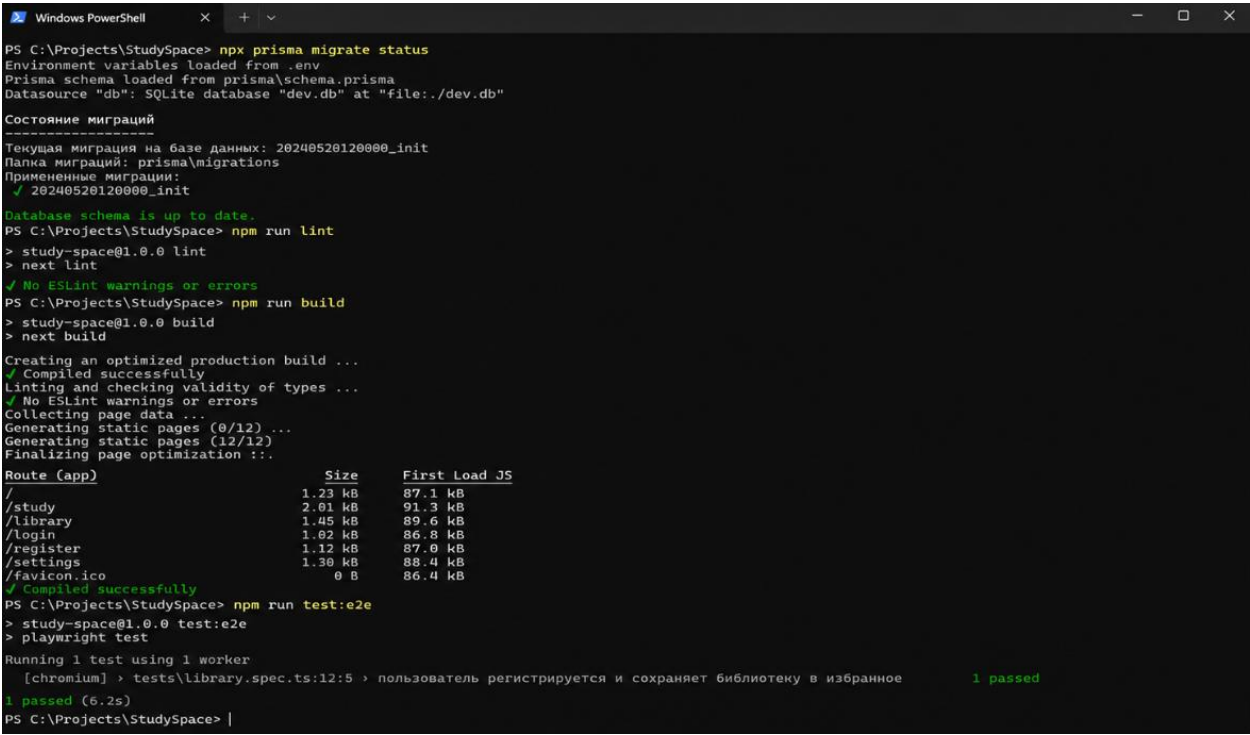
Выполните в указанном порядке:

```
# Проверяем базу и генерируем актуальный Prisma Client.
npx prisma validate
npx prisma migrate status
npx prisma generate

# Проверяем статический анализ и чистую логику.
npm run lint
npm run test:unit

# Собираем production-приложение до запуска E2E webServer.
npm run build
npm run test:e2e
```

Завершите ручной сценарий галереи, модального окна, карты, меню, регистрации, входа, выхода и избранного.



```
Windows PowerShell
PS C:\Projects\StudySpace> npx prisma migrate status
Environment variables loaded from .env
Prisma schema loaded from prisma\schema.prisma
Datasource "db": SQLite database "dev.db" at "file:./dev.db"

Состояние миграций
-----
Текущая миграция на базе данных: 20240520120000_init
Папка миграций: prisma\migrations
Примененные миграции:
✓ 20240520120000_init

Database schema is up to date.
PS C:\Projects\StudySpace> npm run lint
> study-space@1.0.0 lint
> next lint
✓ No ESLint warnings or errors
PS C:\Projects\StudySpace> npm run build
> study-space@1.0.0 build
> next build

Creating an optimized production build ...
✓ Compiled successfully
Linting and checking validity of types ...
✓ No ESLint warnings or errors
Collecting page data ...
Generating static pages (0/12) ...
Generating static pages (12/12) ...
Finalizing page optimization ...

Route (app)      Size      First Load JS
/                 1.23 kB    87.1 kB
/study           2.01 kB    91.3 kB
/library         1.45 kB    89.6 kB
/login           1.02 kB    86.8 kB
/register        1.12 kB    87.0 kB
/settings        1.30 kB    88.4 kB
/favicon.ico     0 B       86.4 kB
✓ Compiled successfully
PS C:\Projects\StudySpace> npm run test:e2e
> study-space@1.0.0 test:e2e
> playwright test

Running 1 test using 1 worker
[chromium] › tests\library.spec.ts:12:5 › пользователь регистрируется и сохраняет библиотеку в избранное 1 passed
1 passed (6.2s)
PS C:\Projects\StudySpace> |
```

Для отчёта нужны собственные скриншоты: JSON `/api/places` для гостя и пользователя; вкладка Network для register, login, me, favorite и logout; галерея до и после перезагрузки; соответствующая строка FavoritePlace в PostgreSQL; успешный Vitest; успешный Playwright; итоговые lint, build и migrate status. Скройте пароли, cookie, JWT, AUTH\_SECRET и DATABASE\_URL.

Если селекторы, подписи форм или маршрут галереи в собственном проекте отличаются, адаптируйте Playwright-тест к фактическому доступному интерфейсу. Нельзя менять UI только ради копирования селекторов из приложения; тест должен описывать собственный пользовательский сценарий.

## 6. Требования к результату

После выполнения работы студент должен предоставить проект, в котором:

- клиент не импортирует статический studyPlaces как источник данных;
- GET /api/places читает PostgreSQL через Prisma;
- DTO не содержит passwordHash и лишние поля;
- Decimal-координаты преобразуются перед JSON;
- isFavorite вычисляется для проверенного пользователя;
- DemoAuthProvider удалён из рабочей конфигурации;
- AuthProvider восстанавливает сессию через /api/auth/me;
- register, login и logout подключены к формам;
- галерея имеет loading, error, empty и success состояния;
- запрос отменяется в cleanup;
- избранное сохраняется в FavoritePlace;
- перезагрузка страницы не сбрасывает авторизацию и избранное;
- гостевой UI не заменяет серверную авторизацию;
- модальное окно, карта, меню и адаптивность сохранены;
- unit tests выполняются независимо от базы;
- E2E проверяет реальный production-сервер;
- тестовый пользователь удаляется после сценария;
- обработаны основные отрицательные статусы;
- prisma migrate status, lint, unit tests, build и e2e завершаются успешно.

## 7. Содержание отчёта

Отчёт должен содержать:

- название и цель работы;
- схему клиент → API → Prisma → PostgreSQL;
- таблицу или список HTTP-контрактов и статусов;
- описание UserDTO и StudyPlaceDTO;
- код /api/places;
- код AuthProvider;
- фрагменты подключённых login и register форм;
- код загрузки галереи и обработки состояний;
- код server toggle избранного;
- доказательство сохранения после reload;
- скриншоты Network без секретов;
- SQL-проверку FavoritePlace;
- матрицу ручных положительных и отрицательных сценариев;
- vitest.config.mts и unit tests;
- playwright.config.ts и E2E test;

- объяснение очистки тестовых данных;
- результаты validate, migrate status, lint, unit, build и e2e;
- результаты повторной ручной проверки ЛР3–ЛР4;
- ответы на контрольные вопросы;
- итоговый вывод по всему курсу.

Контрольные вопросы:

- Как проходит запрос от Client Component до PostgreSQL?
- Что такое HTTP-контракт?
- Где располагается Route Handler в App Router?
- Какие HTTP-методы поддерживает route.ts?
- Почему response.ok нужно проверять вручную?
- Что такое DTO?
- Почему UserDTO не должен повторять всю строку User?
- Зачем Decimal преобразуется перед JSON?
- Где вычисляется isFavorite и почему?
- Как httpOnly cookie участвует в fetch-запросе?
- Чем loading отличается от empty?
- Для чего нужен AbortController?
- Почему AuthProvider имеет состояние loading?
- Чем пессимистичное обновление отличается от оптимистичного?
- Почему userId нельзя принимать от клиента для избранного?
- Чем unit test отличается от integration и E2E?
- Почему async Server Components удобнее проверять E2E?
- Что проверяет Vitest в этой работе?
- Что проверяет Playwright?
- Для чего нужны baseUrl и webServer?
- Почему E2E выполняется после production build?
- Почему выбран один worker?
- Зачем тесту уникальный email?
- Почему тестовые данные нужно удалять?
- Почему waitFotTimeout является плохой синхронизацией?
- Какие проверки остаются ручными?
- Почему проходящие тесты не доказывают полную безопасность?

## 8. Источники

Теоретическая часть подготовлена по официальной документации Next.js, Prisma ORM, MDN, Vitest и Playwright. Набор DTO, HTTP-статусов, селекторы учебного UI, один worker и итоговая последовательность проверок являются методикой курса.

- Next.js: Route Handlers — <https://nextjs.org/docs/app/getting-started/route-handlers>
- Next.js: route.js — <https://nextjs.org/docs/app/api-reference/file-conventions/route>

- Next.js: Authentication — <https://nextjs.org/docs/app/guides/authentication>
- Next.js: cookies — <https://nextjs.org/docs/app/api-reference/functions/cookies>
- Next.js: Testing — <https://nextjs.org/docs/app/guides/testing>
- Next.js: Playwright — <https://nextjs.org/docs/app/guides/testing/playwright>
- Next.js: Vitest — <https://nextjs.org/docs/app/guides/testing/vitest>
- Next.js: Environment Variables — <https://nextjs.org/docs/app/guides/environment-variables>
- Prisma: CRUD — <https://www.prisma.io/docs/orm/v6/prisma-client/queries/crud>
- Prisma: Transactions and batch queries — <https://www.prisma.io/docs/orm/v6/prisma-client/queries/transactions>
- Prisma: Testing — <https://www.prisma.io/docs/orm/v6/prisma-client/testing>
- MDN: Fetch API — [https://developer.mozilla.org/en-US/docs/Web/API/Fetch\\_API](https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API)
- MDN: Using Fetch — [https://developer.mozilla.org/en-US/docs/Web/API/Fetch\\_API/Using\\_Fetch](https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch)
- MDN: HTTP response status codes — <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status>
- Vitest: Getting Started — <https://vitest.dev/guide/>
- Vitest: Writing Tests — <https://vitest.dev/guide/learn/writing-tests>
- Playwright: Writing tests — <https://playwright.dev/docs/writing-tests>
- Playwright: Locators — <https://playwright.dev/docs/locators>
- Playwright: Assertions — <https://playwright.dev/docs/test-assertions>
- Playwright: Web server — <https://playwright.dev/docs/test-webserver>

# 1. Структура новых и изменённых файлов

# Показывает API, клиентскую интеграцию и автоматические тесты.

```
study-space/
├── app/
│   ├── api/
│   │   ├── auth/
│   │   │   └── ...
│   │   ├── favorites/[placeId]/route.ts
│   │   └── places/route.ts
│   ├── auth/
│   │   ├── login/page.tsx
│   │   └── register/page.tsx
│   ├── gallery/
│   │   └── _components/PlacesGallery.tsx
│   ├── providers/AuthProvider.tsx
│   └── layout.tsx
├── tests/
│   ├── e2e/auth-and-favorites.spec.ts
│   └── unit/validation.test.ts
├── playwright.config.ts
├── vitest.config.mts
└── package.json
```

## 2. Файл app/api/places/route.ts

```
// Возвращает учебные места и персональное состояние избранного.

import { NextResponse } from "next/server";
import { getCurrentUser } from "@app/lib/current-user";
import { prisma } from "@app/lib/prisma";

// Ответ зависит от cookie текущего запроса и не является общим кэшем.
export const dynamic = "force-dynamic";

export async function GET() {
  try {
    // Гость получает места без персональных связей.
    const user = await getCurrentUser();

    // Читаем предметные данные и только нужные favorite placeId.
    const [places, favorites] = await Promise.all([
      prisma.studyPlace.findMany({ orderBy: { title: "asc" } }),
      user
        ? prisma.favoritePlace.findMany({
            where: { userId: user.id },
            select: { placeId: true },
          })
        : Promise.resolve([]),
    ]);

    // Set обеспечивает быструю проверку каждого места.
    const favoriteIds = new Set(favorites.map((item) => item.placeId));

    // Формируем стабильный DTO без внутренних временных полей.
    const result = places.map((place) => ({
      id: place.id,
      slug: place.slug,
      title: place.title,
```

```

        summary: place.summary,
        address: place.address,
        image: place.image,
        imageAlt: place.imageAlt,
        latitude: Number(place.latitude),
        longitude: Number(place.longitude),
        isFavorite: favoriteIds.has(place.id),
    }));

    return NextResponse.json({ places: result });
  } catch {
    // Не раскрываем клиенту объект ошибки подключения или DATABASE_URL.
    return NextResponse.json(
      { error: "Не удалось загрузить учебные места" },
      { status: 500 },
    );
  }
}

```

### 3. Файл app/providers/AuthProvider.tsx

```

// Синхронизирует клиентский интерфейс с серверной cookie-сессией.

"use client";

import {
  createContext,
  useCallback,
  useContext,
  useEffect,
  useMemo,
  useState,
  type ReactNode,
} from "react";

export type UserDTO = {
  id: string;
  name: string;
  email: string;
};

type AuthStatus = "loading" | "authenticated" | "guest";

type AuthContextValue = {
  user: UserDTO | null;
  status: AuthStatus;
  refresh: () => Promise<void>;
  login: (email: string, password: string) => Promise<void>;
  register: (input: {
    name: string;
    email: string;
    password: string;
    confirmPassword: string;
  }) => Promise<void>;
  logout: () => Promise<void>;
};

const AuthContext = createContext<AuthContextValue | null>(null);

// Читаем безопасное сообщение из согласованного JSON-ответа.
async function readError(response: Response): Promise<string> {
  const payload = (await response.json().catch(() => null)) as
    | { error?: string; errors?: string[] }
    | null;

  return payload?.errors?.[0] ?? payload?.error ?? "Ошибка запроса";
}

```

```

}

export default function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState<UserDTO | null>(null);
  const [status, setStatus] = useState<AuthStatus>("loading");

  // Восстанавливаем состояние из серверной cookie, не читая её на клиенте.
  const fetchCurrentUser = useCallback(async (): Promise<UserDTO | null> => {
    const response = await fetch("/api/auth/me", {
      cache: "no-store",
      credentials: "same-origin",
    });

    if (!response.ok) {
      return null;
    }

    const payload = (await response.json()) as { user: UserDTO | null };
    return payload.user;
  }, []);

  const refresh = useCallback(async () => {
    const currentUser = await fetchCurrentUser();

    setUser(currentUser);
    setStatus(currentUser ? "authenticated" : "guest");
  }, [fetchCurrentUser]);

  useEffect(() => {
    void fetchCurrentUser().then((currentUser) => {
      setUser(currentUser);
      setStatus(currentUser ? "authenticated" : "guest");
    });
  }, [fetchCurrentUser]);

  const login = useCallback(async (email: string, password: string) => {
    const response = await fetch("/api/auth/login", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      credentials: "same-origin",
      body: JSON.stringify({ email, password }),
    });

    if (!response.ok) throw new Error(await readError(response));

    const payload = (await response.json()) as { user: UserDTO };
    setUser(payload.user);
    setStatus("authenticated");
  }, []);

  const register = useCallback(async (input: {
    name: string;
    email: string;
    password: string;
    confirmPassword: string;
  }) => {
    const response = await fetch("/api/auth/register", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      credentials: "same-origin",
      body: JSON.stringify(input),
    });

    if (!response.ok) throw new Error(await readError(response));

    const payload = (await response.json()) as { user: UserDTO };
    setUser(payload.user);
    setStatus("authenticated");
  }, []);

```

```

}, []);

const logout = useCallback(async () => {
  const response = await fetch("/api/auth/logout", {
    method: "POST",
    credentials: "same-origin",
  });

  if (!response.ok) throw new Error(await readError(response));
  setUser(null);
  setStatus("guest");
}, []);

// Стабилизируем объект контекста для дочерних Client Components.
const value = useMemo(
  () => ({ user, status, refresh, login, register, logout }),
  [user, status, refresh, login, register, logout],
);

return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

export function useAuth(): AuthContextValue {
  const value = useContext(AuthContext);
  if (!value) throw new Error("useAuth must be used inside AuthProvider");
  return value;
}

```

## 4. Обновлённый файл app/layout.tsx

```

// Подключает реальный AuthProvider вместо учебного DemoAuthProvider.

import type { Metadata } from "next";
import type { ReactNode } from "react";
import AuthProvider from "../providers/AuthProvider";
import BurgerMenu from "../ui/BurgerMenu/BurgerMenu";
import "../globals.css";

export const metadata: Metadata = {
  title: "StudySpace",
  description: "Учебные места для самостоятельной работы",
};

export default function RootLayout({ children }: { children: ReactNode }) {
  // Провайдер обслуживает только клиентские потребители auth-состояния.
  return (
    <html lang="ru">
      <body>
        <AuthProvider>
          <BurgerMenu />
          {children}
        </AuthProvider>
      </body>
    </html>
  );
}

```

## 5. Файл app/auth/login/page.tsx

```

// Подключает форму входа к POST /api/auth/login через AuthProvider.

"use client";

import Link from "next/link";

```

```

import { useRouter } from "next/navigation";
import { useState, type FormEvent } from "react";
import { useAuth } from "@app/providers/AuthProvider";

export default function LoginPage() {
  const router = useRouter();
  const { login } = useAuth();
  const [error, setError] = useState("");
  const [pending, setPending] = useState(false);

  async function handleSubmit(event: FormEvent<HTMLFormElement>) {
    event.preventDefault();
    setError("");
    setPending(true);

    const form = new FormData(event.currentTarget);

    try {
      // Передаём пароль серверу только в теле HTTPS-запроса production-среды.
      await login(String(form.get("email")) ?? "", String(form.get("password")) ?? "");
      router.push("/gallery");
    } catch (value) {
      setError(value instanceof Error ? value.message : "Не удалось войти");
    } finally {
      setPending(false);
    }
  }

  return (
    <main className="form-page">
      <h1>Вход</h1>
      <form className="auth-form" onSubmit={handleSubmit}>
        <label htmlFor="email">Email</label>
        <input name="email" type="email" autoComplete="email" required />

        <label htmlFor="password">Пароль</label>
        <input
          id="password"
          type="password"
          autoComplete="current-password"
          minLength={8}
          required
        />

        {error ? <p role="alert" className="form-error">{error}</p> : null}
        <button type="submit" disabled={pending}>
          {pending ? "Вход..." : "Войти"}
        </button>
      </form>
      <p>
        Нет учебной записи? <Link href="/auth/register">Регистрация</Link>
      </p>
    </main>
  );
}

```

## 6. Файл app/auth/register/page.tsx

```

// Подключает форму регистрации к POST /api/auth/register.

"use client";

import { useRouter } from "next/navigation";
import { useState, type FormEvent } from "react";
import { useAuth } from "@app/providers/AuthProvider";

```

```

export default function RegisterPage() {
  const router = useRouter();
  const { register } = useAuth();
  const [error, setError] = useState("");
  const [pending, setPending] = useState(false);

  async function handleSubmit(event: FormEvent<HTMLFormElement>) {
    event.preventDefault();
    setError("");
    setPending(true);

    const form = new FormData(event.currentTarget);

    try {
      // Формируем контракт регистрации из полей доступной HTML-формы.
      await register({
        name: String(form.get("name") ?? ""),
        email: String(form.get("email") ?? ""),
        password: String(form.get("password") ?? ""),
        confirmPassword: String(form.get("confirmPassword") ?? ""),
      });
      router.push("/gallery");
    } catch (value) {
      setError(value instanceof Error ? value.message : "Не удалось зарегистрироваться");
    } finally {
      setPending(false);
    }
  }

  return (
    <main className="form-page">
      <h1>Регистрация</h1>
      <form className="auth-form" onSubmit={handleSubmit}>
        <label htmlFor="name">Имя пользователя</label>
        <input name="name" autoComplete="name" required />
        <label htmlFor="email">Email</label>
        <input name="email" type="email" autoComplete="email" required />
        <label>Пароль</label>
        <input
          name="password"
          type="password"
          autoComplete="new-password"
          minLength={8}
          required
        />
        <label>Повторите пароль</label>
        <input
          name="confirmPassword"
          type="password"
          autoComplete="new-password"
          minLength={8}
          required
        />

        {error ? <p role="alert">{error}</p> : null}
        <button type="submit" disabled={pending}>
          {pending ? "Регистрация..." : "Зарегистрироваться"}
        </button>
      </form>
    </main>
  );
}

```

## 7. Фрагмент

# app/gallery/\_components/PlacesGallery.tsx

```
// Загружает реальные места и сохраняет избранное через серверный API.

"use client";

import Image from "next/image";
import Link from "next/link";
import { useCallback, useEffect, useRef, useState } from "react";
import PhotoMap from "../PhotoMap";
import PhotoModal from "../PhotoModal";
import { useAuth } from "@app/providers/AuthProvider";
import styles from "../PlacesGallery.module.css";

export type StudyPlaceDTO = {
  id: string;
  slug: string;
  title: string;
  summary: string;
  address: string;
  image: string;
  imageAlt: string;
  latitude: number;
  longitude: number;
  isFavorite: boolean;
};

export default function PlacesGallery() {
  const { status, refresh } = useAuth();
  const [places, setPlaces] = useState<StudyPlaceDTO[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");
  const [pendingId, setPendingId] = useState<string | null>(null);
  const [reloadKey, setReloadKey] = useState(0);

  const [selected, setSelected] = useState<StudyPlaceDTO | null>(null);
  const triggerRef = useRef<HTMLButtonElement | null>(null);

  const openFromMap = useCallback((place: StudyPlaceDTO) => {
    triggerRef.current = null;

    const selectedPlace = places.find((item) => item.id === place.id);

    if (selectedPlace) {
      setSelected(selectedPlace);
    }
  }, [places]);

  function closeModal() {
    setSelected(null);
  }

  // Повторная загрузка вызывается кнопкой после сетевой ошибки.
  const retry = useCallback(() => setReloadKey((value) => value + 1), []);

  useEffect(() => {
    const controller = new AbortController();

    async function loadPlaces() {
      setLoading(true);
      setError("");

      try {
        const response = await fetch("/api/places", {

```

```

        cache: "no-store",
        credentials: "same-origin",
        signal: controller.signal,
    });

    if (!response.ok) throw new Error("Не удалось загрузить учебные места");

    const payload = (await response.json()) as { places: StudyPlaceDTO[] };
    setPlaces(payload.places);
  } catch (value) {
    // Игнорируем ожидаемую отмену при размонтировании компонента.
    if (value instanceof DOMException && value.name === "AbortError") return;
    setError(value instanceof Error ? value.message : "Ошибка загрузки");
  } finally {
    if (!controller.signal.aborted) setLoading(false);
  }
}

void loadPlaces();

// Отменяем запрос, если компонент больше не используется.
return () => controller.abort();
}, [reloadKey, status]);

async function toggleFavorite(place: StudyPlaceDTO) {
  if (status !== "authenticated") return;

  setPendingId(place.id);
  setError("");

  try {
    const response = await fetch(`/api/favorites/${place.id}`, {
      method: place.isFavorite ? "DELETE" : "POST",
      credentials: "same-origin",
    });

    if (response.status === 401) {
      // Синхронизируем UI, если сервер больше не принимает сессию.
      await refresh();
      throw new Error("Сеанс завершён. Выполните вход снова.");
    }

    if (!response.ok) throw new Error("Не удалось изменить избранное");

    // Меняем только карточку, изменение которой подтвердил сервер.
    setPlaces((current) =>
      current.map((item) =>
        item.id === place.id
          ? { ...item, isFavorite: !item.isFavorite }
          : item,
      ),
    );
  } catch (value) {
    setError(value instanceof Error ? value.message : "Ошибка избранного");
  } finally {
    setPendingId(null);
  }
}

if (loading) return <p role="status">Загрузка учебных мест...</p>;

if (error && places.length === 0) {
  return (
    <div role="alert">
      <p>{error}</p>
      <button type="button" onClick={retry}>Повторить</button>
    </div>
  );
}

```

```

    }

    if (places.length === 0) return <p>Учебные места пока не добавлены.</p>;

    return (
      <>
        {error ? <p role="alert">{error}</p> : null}

        <div className={styles.grid}>
          {places.map((place) => (
            <article className={styles.card} key={place.id}>
              <div className={styles.imageWrap}>
                <Image
                  src={place.image}
                  alt={place.imageAlt}
                  fill
                  sizes="(max-width: 48rem) 100vw, 33vw"
                  className={styles.image}
                />
              </div>

              <h2 className={styles.title}>{place.title}</h2>
              <p className={styles.summary}>{place.summary}</p>

              {status === "authenticated" ? (
                <button
                  type="button"
                  disabled={pendingId === place.id}
                  aria-pressed={place.isFavorite}
                  aria-label={` ${
                    place.isFavorite
                      ? "Удалить из избранного"
                      : "Добавить в избранное"
                    }`}
                  onClick={() => void toggleFavorite(place)}
                >
                  {place.isFavorite ? "В избранном" : "В избранное"}
                </button>
              ) : (
                <Link href="/auth/login">Войдите, чтобы добавить в избранное</Link>
              )}
            </article>
          ))}
        </div>

        {/* Передайте places в PhotoMap и выбранный объект в PhotoModal из ЛР3. */}
      </>
    );
  }
}

```

## 8. Обновление PhotoModal и BurgerMenu

```

// В обоих компонентах заменяем учебный провайдер реальным.
import { useAuth } from "@app/providers/AuthProvider";

// Получаем подтверждённое клиентское представление серверной сессии.
const { user, status, logout } = useAuth();

// Скрываем персональные действия, пока refresh ещё выполняется.
const authReady = status !== "loading";

```

Сохраните доступность, фокус, Escape, карту и CSS из ЛР3–ЛР4. Кнопка модального окна должна вызывать тот же `toggleFavorite`, а не создавать второй независимый источник состояния.

## 9. Файл vitest.config.mts

```
// Настраивает быстрые unit tests чистой серверной логики.

import { defineConfig } from "vitest/config";
import tsconfigPaths from "vite-tsconfig-paths";

export default defineConfig({
  // Поддерживаем алиасы путей из tsconfig.json.
  plugins: [tsconfigPaths()],
  test: {
    // Чистая валидация не требует DOM или браузера.
    environment: "node",
    include: ["tests/unit/**/*.test.ts"],
  },
});
```

## 10. Файл tests/unit/validation.test.ts

```
// Проверяет чистую нормализацию и server-side validation без базы.

import { describe, expect, test } from "vitest";
import {
  normalizeEmail,
  validateRegistration,
} from "@app/lib/validation";

describe("normalizeEmail", () => {
  test("удаляет пробелы и приводит email к нижнему регистру", () => {
    expect(normalizeEmail(" Student@Example.TEST ")).toBe(
      "student@example.test",
    );
  });
});

describe("validateRegistration", () => {
  test("принимает корректный набор полей", () => {
    const result = validateRegistration({
      name: "Студент",
      email: "student@example.test",
      password: "StudyPass_2026!",
      confirmPassword: "StudyPass_2026!",
    });

    expect(result.errors).toEqual([]);
    expect(result.data?.email).toBe("student@example.test");
  });

  test("отклоняет короткий пароль", () => {
    const result = validateRegistration({
      name: "Студент",
      email: "student@example.test",
      password: "short",
      confirmPassword: "short",
    });

    expect(result.data).toBeNull();
    expect(result.errors).toContain(
      "Пароль должен содержать не менее 8 символов",
    );
  });

  test("отклоняет несовпадающее подтверждение", () => {
    const result = validateRegistration({
```

```

    name: "Студент",
    email: "student@example.test",
    password: "StudyPass_2026!",
    confirmPassword: "Different_2026!",
  });

  expect(result.errors).toContain("Пароли не совпадают");
});
});

```

## 11. Файл playwright.config.ts

```

// Запускает E2E против production-сборки StudySpace.

import "dotenv/config";
import { defineConfig, devices } from "@playwright/test";

export default defineConfig({
  testDir: "./tests/e2e",
  // Общая локальная база проверяется последовательно.
  workers: 1,
  use: {
    baseURL: "http://127.0.0.1:3000",
    trace: "on-first-retry",
    ...devices["Desktop Chrome"],
  },
  webServer: {
    // Перед test:e2e уже должна существовать npm run build.
    command: "npm run start",
    url: "http://127.0.0.1:3000",
    reuseExistingServer: !process.env.CI,
    timeout: 120_000,
  },
});

```

## 12. Файл tests/e2e/auth-and-favorites.spec.ts

```

// Проверяет регистрацию, cookie-сессию и сохранение избранного после reload.

import "dotenv/config";
import { PrismaClient } from "@prisma/client";
import { expect, test } from "@playwright/test";

const prisma = new PrismaClient();
let testEmail = "";

test.afterAll(async () => {
  // Удаляем только пользователя, созданного этим тестом.
  if (testEmail) {
    await prisma.user.deleteMany({ where: { email: testEmail } });
  }
  await prisma.$disconnect();
});

test("пользователь регистрируется и сохраняет библиотеку в избранное", async ({
  page,
}) => {
  // Уникальный email исключает конфликт с предыдущим локальным запуском.
  testEmail = `e2e-${Date.now()}@example.test`;

  await page.goto("/auth/register");
  await page.getByLabel("Имя пользователя").fill("E2E студент");
  await page.getByLabel("Email").fill(testEmail);
  await page.getByLabel("Пароль", { exact: true }).fill("StudyPass_2026!");

```

```

await page.getByLabel("Повторите пароль").fill("StudyPass_2026!");
await page.getByRole("button", { name: "Зарегистрироваться" }).click();

// После успешного API-ответа форма переводит пользователя в галерею.
await expect(page).toHaveURL(/\/gallery$/);

const addFavorite = page.getByRole("button", {
  name: "Добавить в избранное: Научная библиотека",
});
await addFavorite.click();

// Сервер подтвердил изменение, поэтому доступно действие удаления.
await expect(
  page.getByRole("button", {
    name: "Удалить из избранного: Научная библиотека",
  }),
).toBeVisible();

// Перезагрузка повторно читает cookie и FavoritePlace из PostgreSQL.
await page.reload();
await expect(
  page.getByRole("button", {
    name: "Удалить из избранного: Научная библиотека",
  }),
).toBeVisible();

// Завершаем сессию через тот же browser context.
await page.request.post("/api/auth/logout");
await page.reload();
await expect(
  page.getByRole("link", { name: "Войдите, чтобы добавить в избранное" }).first(),
).toBeVisible();
});

```

Доступное имя кнопки в компоненте и Playwright-селекторе должно совпадать. При изменении текста интерфейса согласованно обновите тест.

## 13. Скрипты package.json

```

{
  "scripts": {
    "test:unit": "vitest run",
    "test:e2e": "playwright test",
    "check:final": "npm run lint && npm run test:unit && npm run build && npm run
test:e2e"
  }
}

```

JSON не поддерживает комментарии. `test:unit` проверяет чистую логику, `test:e2e` запускает браузерный сценарий, а `check:final` объединяет итоговую последовательность после проверки миграций.

## 14. Команды установки и итоговой проверки

```

# Устанавливаем тестовые инструменты.
npm install -D vitest vite-tsconfig-paths @playwright/test
npx playwright install chromium

# Проверяем актуальность базы и Prisma Client.

```

```
npx prisma validate
npx prisma migrate status
npx prisma generate
```

```
# Запускаем проверки от быстрых к более длительным.
npm run lint
npm run test:unit
npm run build
npm run test:e2e
```

## 15. Что студент должен заменить

- StudyPlaceDTO и поля API — на собственную предметную сущность;
- URL /gallery — на фактический маршрут собственного каталога;
- подписи форм и кнопок — на доступные имена собственного интерфейса;
- текст «Научная библиотека» — на устойчивый seed-объект своего проекта;
- CSS и разметку карточки — на дизайн из ЛР3–ЛР4;
- тестовые данные — на собственные, сохранив уникальность и очистку;
- скриншоты, Network и SQL-результаты — только собственные; секреты не публиковать и серверную авторизацию не ослаблять.